MODULE

07

Team workflows

How real teams ship: GitHub flow, pull requests, reviews, commit conventions and branching strategies.

YOU WILL LEARN

GitHub flow	`01_github_flow.py`
Conventional Commits	`02_conventional_commits.py`
Branching strategies	`03_branching_strategies.md`
Pull requests & reviews	`04_pull_requests_and_reviews.md`
Exercise: great commits	`ex07_conventional_commits.py`
Interview Q&A	`INTERVIEW.md`

WHY WORKFLOWS

Ten developers, one main branch

THE PROBLEM

Without rules, people push half-finished work to main, production breaks on Friday evening, and nobody knows which change caused it.

THE IDEA

A workflow is the team's agreement: branch per task, pull request with CI and review, then merge into a protected main.

PICTURE IT GITHUB FLOW

The GitHub flow loop

🍳 A restaurant kitchen: each dish (branch) is prepared at its own station and only reaches the customer (main) after the head chef tastes it (review + CI).



TRY IT MERGE BUTTONS

”Squash and merge” = one commit per PR

```
$ git merge --squash feature/wishlist
Squash commit -- not updating HEAD
Automatic merge went well; stopped before committing as requested
$ git commit -qm "Add wishlist (#42)"
$ git log --oneline --graph
* 878cf6e Add wishlist (#42)
* 55c1d7e Add footer
* 3d365c0 Initial commit
```

1. The 2 branch commits become 1 on main.
2. Easy to read; easy to revert a whole PR.
3. ”Create a merge commit” = **--no-ff**; ”Rebase and merge” = rebase + fast-forward.

STEP BY STEP MERGE BUTTONS

The three GitHub merge buttons

BUTTON	HISTORY ON MAIN	CHOOSE WHEN
`Create a merge commit`	all commits + a merge commit	you want the full branch story
`Squash and merge`	one commit per PR	most teams: clean main, easy reverts
`Rebase and merge`	linear, every commit kept	commits are already clean and meaningful

TRY IT CONVENTIONAL COMMITS

Commit messages that tools can read

```
feat(cart): add coupon codes          -> minor  1.2.0 -> 1.3.0
fix(cart): round totals to 2 decimals -> patch  1.2.0 -> 1.2.1
docs: explain how to run the shop
refactor(search): extract query parser
test(auth): cover expired tokens
chore: bump django to 5.1
feat(auth)!: require two-factor login -> major  1.2.0 -> 2.0.0
```

1. **type(scope): summary** in the imperative.
2. Tools turn the log into **release notes** and pick the next version.
3. The body explains **why**; the diff already shows what.

STEP BY STEP BRANCHING STRATEGIES

GitHub flow vs Git flow vs trunk-based

	GITHUB FLOW	GIT FLOW	TRUNK-BASED
Long-lived branches	main	main + develop	main
Branch lifetime	days	weeks	hours
Releases	every merge	planned versions	continuous
Unfinished work	on a branch	on a branch	feature flags
Best for	web apps, services	versioned products	strong-CI teams

Most web teams: **GitHub flow**. Say which you'd choose and why.

STEP BY STEP PULL REQUESTS

Protect main

SETTING	EFFECT
Branch protection / rulesets	no direct pushes, no force pushes to main
Required reviews	e.g. 1 approval; stale approvals dismissed on new commits
Required status checks	CI must be green
CODEOWNERS	`/payments/ @shop/payments-team` auto-requested
Require linear history	only squash / rebase merges

COMMON MISTAKE PULL REQUESTS

The 2,000-line pull request

✗ WRONG

```
Title: Update
Description: (empty)
+2,143 -871 in 46 files
"LGTM" after 3 minutes
```

Nobody can review that properly. Bugs slip through, and the branch has drifted for weeks: merge conflicts everywhere.

✓ RIGHT

```
Title: Add coupon codes to checkout
What / Why (SHOP-142) / How I tested
+180 -12 in 5 files, CI green
```

Small PRs, a clear description, green CI. Big features: several PRs behind a feature flag.

IN REAL JOBS TEAMWORK

Your daily team routine

Start a ticket

```
git switch main && git pull
git switch -c feature/SHOP-142-coupons
```

Open the PR

```
git push -u origin feature/SHOP-142-coupons
# GitHub: Compare & pull request
```

Answer the review

```
git commit -m "fix(cart): handle empty coupon"
git push
```

After the merge

```
git switch main && git pull
git branch -d feature/SHOP-142-coupons
```

INTERVIEW SAY THIS IN THE INTERVIEW

“Describe your team’s Git workflow.”

- **GitHub flow**: a short-lived branch per ticket from the latest main.
- Push, open a **PR** with a description; **CI** runs; one or two reviewers approve.
- **Squash-merge** into a **protected** main, which deploys automatically.
- Big features land in small PRs behind **feature flags**.

> **Follow-up they ask next:** ”GitHub flow vs Git flow?” (Git flow adds develop/release/hotfix branches for versioned releases; heavier, more merge pain).

RECAP REMEMBER THESE

Module 7 in 5 lines

1. Branch → PR → CI + review → merge → deploy.
2. Merge buttons: merge commit, **squash**, rebase.
3. **type(scope): summary** commits power changelogs and versions.
4. GitHub flow for most web teams; Git flow for versioned releases.
5. Protect main: reviews, checks, CODEOWNERS.

PRACTICE NOW

Do exercises/ex07_conventional_commits.py: three changes, three clean commits.